Strata — Hub & Child Guide

For knowledge owners and authors

strata-kb v1.34.0 · 2026-11-01

1. Who this guide is for

You own knowledge, or you produce it.

- A **hub** is the place your organisation's knowledge lives and is searched. If you run the hub, you operate the service everyone else reads from, and you are the last gate before anything becomes findable.
- A **child** is an authoring repository. If you work in one, you bring documents in, get them summarized, check them, and publish them to the hub.

Both roles use the same commands and the same four-layer model, so they are covered together. Where a step applies to only one, it says so.

Two other repository kinds publish to the same hub and have their own guides: a `dev` repo publishes knowledge about its own code (`<repo>-code`, `<repo>-svc`), and a `ba` repo publishes its plan (`<repo>-plan`: its missions and ready tickets). As the hub operator you register both like any child (§9.3).

You do not need to read the architecture document first. You do need to understand one rule before anything else:

> **Nothing you author is searchable until a pull request on the hub merges.** Your local store is a drafting desk. The hub is the published world.

---

2. Install

```
pip install "strata-kb[ingest]"
kb --help
```

The `ingest` extra brings the PDF pipeline. Without it every other command still works — useful on a machine that only queries or publishes.

Requirements. Python 3.11 or newer, and Git. If your hub URL carries a credential (the `https://x-access-token:…@github.com/...` shape), you need **Git 2.31 or newer** on every machine and every CI runner. Older Git silently ignores the mechanism that keeps the token out of the clone's config, and every hub command fails with an authentication error. Debian 11 and Ubuntu 20.04 ship below that floor.

Prefer Docker? `kb docker-setup` handles it; see section 9.

---

3. Create the repository

```
mkdir my-kb && cd my-kb && git init
kb init            # interactive: it asks which kind
kb init --kind hub # or non-interactive
```

`kb init` scaffolds for the kind you choose and records it in `.kb/config.yaml`. Re-running it later refreshes skills, templates and workflows while preserving your data — pass `--force` only if you really want the data files reset too.

3.1 Hub configuration

```
# .kb/config.yaml on the hub
kind: hub
repo_id: ops              # this hub's own entry id, if it authors content
asset_store:
 mode: none               # or: {mode: s3, bucket: my-kb-assets}
```

3.2 Child configuration

```
# .kb/config.yaml on a child
kind: child
repo_id: safety           # your entry id under federation/
hub: https://github.com/acme/kb-hub.git
```

`repo_id` becomes the prefix in every citation this repository produces (`safety:handbook §4.2`), so choose something short and durable. Changing it later orphans the old entry on the hub.

> **Keep credentials out of `config.yaml` where you can.** A `hub:` URL with a token in it works, but prefer ssh (`git@github.com:acme/kb-hub.git`) or a public https URL on workstations, and keep the credentialed form for CI.

---

4. Bring a document in

```
kb ingest sources/employee-handbook.pdf \
 --id hr-handbook \
 --tags hr,policy,benefits \
 --revision "2026 edition"
```

Flag	What it does	Advice
`--id`	The document's permanent id	Short, lowercase, hyphenated. It appears in every citation forever.
`--tags`	Labels used to pre-filter search	2–5 tags. Think about how someone would narrow a search, not how you would file the document.
`--revision`	Edition label	Always set it. It appears in every citation and is how a reader knows which edition an answer came from.
`--sections`	Restrict to certain chapters	Use it for a first trial run on a long document.
`--no-bookmarks`	Split by heading pattern instead of the PDF outline	Only if the outline is missing or wrong.
`--no-summarize`	Skip the automatic summarize pass	When you want to inspect the split before spending model calls.

4.1 Where source files go

Source PDFs are usually licensed or confidential. They live in `sources/`, which the scaffolding configures Git to ignore. **Do not commit them.** The knowledge base carries the extracted text under your own control; the source stays where it legally belongs.

4.2 What ingestion produces

```
.kb/hr-handbook/
├── _manifest.yaml              L1 — every section, all "pending"
├── ch4-benefits.md             L2 — tables in place, prose blank
├── ch4-benefits.raw.md         L3 — the full original
└── assets/<sha256>.webp        figures, stored once each
```

The first run on any machine downloads a page-layout model (~500 MB) and caches it, so it is slow once and fast afterwards.

4.3 Read the ingest report

The report is not decoration. It names every judgement call the splitter made: headings it demoted to ordinary text, ids it had to invent, numbered headings that arrived out of order, duplicate ids it renamed, bookmarks it could not match, and the distribution of section sizes.

Skim it before moving on. A document that produced many fallback ids or many out-of-order headings is usually a document whose outline needs `--no-bookmarks` and a heading pattern instead.

4.4 Re-ingesting

- **Whole document** (no `--sections`): a full replace. Every Markdown file of that document is deleted and rewritten — including summaries you have already reviewed.
- **One chapter** (`--sections 6`): only that chapter's files and manifest entries are rewritten. Everything else, including reviewed summaries and their status, is untouched.
- A `--sections` value that matches no heading **deletes** that chapter. The report says so explicitly; read it rather than assume.

4.5 Review the L3 check

After writing L3, `kb ingest` compares every section with the PDF's own text layer and writes what it finds to `.kb/<id>/_l3check.yaml` (commit it). It flags a section when its id is a fallback, it is out of order, text is missing, extra or garbled, a paragraph is broken in the middle, or the pages have no text layer to check. An LLM agent (the configured `llm.model`, Sonnet by default) adds a verdict to each broken-paragraph finding (`--no-llm-review` skips it); it never clears a flag or edits L3.

A document with unconfirmed findings cannot move on: `kb summarize` skips it (exit 1), `kb build` errors, `kb publish` and `kb ci-publish` refuse.

```
kb l3 report hr-handbook                    # findings, evidence, verdicts
kb l3 confirm hr-handbook --section 4.12    # a reviewer checked this one
kb l3 confirm hr-handbook --all --yes       # every flagged section
```

A confirmation holds only while that section's L3 is unchanged; re-ingesting a changed section asks for review again. Documents ingested before 1.31.1 have no check file and are not gated until re-ingested. `_l3check.yaml` is never

sent to the hub.

4.6 Fix the structure of L3 with AI (optional)

`kb ingest` can leave structure problems in L3: a fallback section id, a wrong heading level, a paragraph broken in the middle, a repeated block, a page header or footer left in the text. The L3 AI fix lets an AI assistant repair that structure while a machine checks that it changed nothing else. It is off by default, and L3 stays the PDF's text.

```
kb l3 fix on                                  # l3_fix: enabled in .kb/config.yaml; adds .kb-work/ to
kb ingest source/handbook.pdf --id hr-handbook # writes a baseline in .kb/hr-handbook/_l3fix.yaml; does
kb l3 lint hr-handbook --pdf source/handbook.pdf
# run the kb-l3-fix skill (Claude Code /kb-l3-fix, a Copilot prompt, a Cursor command); it writes drafts
kb l3 fix apply hr-handbook --pdf source/handbook.pdf --assistant claude-code
kb l3 report hr-handbook                      # AI fixes first, each with a git diff hint
kb l3 confirm hr-handbook --section 4.12      # a human checked it
kb summarize
```

- `kb ingest` always writes the scratch file `.kb-work/<id>/sections.json`. With the switch on it also writes the baseline `_l3fix.yaml` and leaves summarizing to you, printing the next steps.
- `kb l3 lint` checks the layout rules R1 to R8: fallback id, duplicate id, heading levels, repeated blocks, page headers and footers, blocks L3 does not allow, an unclosed code fence, a broken paragraph. It is advisory.
- The skill writes one draft per section in `.kb-work/<id>/l3fix/<section-id>.md` (`<!-- replaces: <id> <id> -->` on the first line to merge or reorder sections). It never edits `.kb/` and never confirms.
- `kb l3 fix apply` checks each draft in this order and writes nothing on the first failure: the `replaces` header, `status` (every section of the chapter file must still be `pending`), `tables` (byte for byte, same order), `contract` (the lint rules), `words` (word by word against the PDF's text layer: no word the PDF does not have at that place, no PDF text lost, no partial restore, no text copied from a neighbour; a new section id must be text the PDF shows) and `ids`. Each line reads `§<name> applied → <new ids>`, `§<name> unchanged` or `§<name> rejected: <check>: <detail>`.
- For a scanned section or a broken font, `kb l3 fix pages <id> --pdf <pdf> --page N` renders PNG pages to `.kb-work/<id>/pages/`, and `apply ... --from-image <id>` skips the word check. The fix is marked unverified and `kb l3 report` lists it first: read it against the PDF.
- The gate: while an AI fix awaits review, or L3 was edited or removed outside `kb l3 fix apply`, `kb summarize` skips the document, `kb build` errors, and `kb publish` and `kb ci-publish` refuse. It reads `_l3fix.yaml`, not the switch. `kb l3 confirm` clears both `_l3check.yaml` and `_l3fix.yaml`. Commit `_l3fix.yaml`; it is never sent to the hub. Bump the CI pin to `strata-kb==1.33.0`: an older CI ignores it and does not gate.

The AI never confirms. The machine checks tables and words, not whether a heading level or a merged paragraph is what the author meant: a reviewer reads the `git diff` against the PDF.

---

5. Summarize

This is the only step a model touches.

`kb ingest` runs it automatically, calling a headless LLM CLI it finds on your `PATH` (`claude`, then `copilot`). You can pin one with `--llm`, or skip it with `--no-summarize`.

```
kb status                   # what is still pending
kb summarize hr-handbook    # run or retry
```

If no LLM CLI is installed, sections simply stay `pending`. Open your coding agent and run `/kb-summarize` instead — it reads `kb status`, fans the pending sections out to parallel read-only sub-agents, and merges their output under the same rules.

5.1 The rules the model is held to

Rule	Why
Write in the source document's language	A summary in another language shares no vocabulary with its original, and keyword search stops matching it
Never rephrase codes, field names, numbers, units or cross-references	These are the content people look the document up *for*
Never touch tables	Tables are machine-copied and automatically verified; a model editing one is a build failure
Prefer the original wording over invention	An unhelpful-but-faithful summary is recoverable; a confident wrong one is not

5.2 Re-running

```
kb summarize hr-handbook --redo --section 4.12       # one section
kb summarize hr-handbook --redo --dry-run            # see the plan first
kb summarize --redo --all                            # the whole store
kb summarize hr-handbook --print-prompt --section 4.12
```

`--redo` skips sections a human has already reviewed unless you add `--include-reviewed --yes`. Sections with very little prose are copied verbatim without a model call at all.

---

6. Check the work

```
kb build
```

```
kb build: OK
```

```
[error] hr-handbook §4.12: L2 table does not match L3 table
```

`kb build` runs three checks:

1. **Nothing left blank.** No `TODO` markers, no empty summaries.
2. **Every table matches its original**, in both directions. This is the one check that cannot be waived.
3. **Quality rules** — summary length relative to the original, no sentence that merely transcribes its own table, no invented uppercase codes, sufficient vocabulary overlap with the original, L1 within 25 words, L0 within 30. These are warnings by default; `--strict` makes them errors.

It also fails if an original changed after its summary was written, or a summary changed after it was approved. And it never writes the manifest while reporting an error, so a failed build leaves nothing half-updated.

```
kb build --allow-pending   # validate finished work while the rest is in progress
kb build --strict          # what `kb approve` requires
```

6.1 Fixing a table mismatch

Open the section's `.raw.md` (L3), copy the table exactly, and paste it over the one in the `.md` (L2). Do not retype it, and do not reformat it — the comparison tolerates alignment and whitespace, nothing else.

---

7. Approve (optional but recommended)

```
kb approve hr-handbook
kb approve hr-handbook --by "Jane Smith <jane@acme.com>"
```

`kb approve` requires a clean working tree for that document and a passing `kb build --strict`. It records who approved each section and when, plus a hash of the summary they approved — so a later edit to an approved summary is detected rather than assumed benign.

Approval is a human act and is never automatic. The hub pull request remains the gate that actually controls publication; approval is how you mark per-section expert sign-off inside it.

---

8. Review a change

If you are reviewing rather than authoring, this is your whole job. No commands — read the diff.

- [ ] **Read the new prose** — the `.md` files, not `.raw.md`. Does it match your understanding of the subject?
- [ ] **Compare against the source.** Anything important omitted? Anything present that is not in the original?
- [ ] **Check every code, identifier, number and unit.** They must be exactly as in the source.
- [ ] **Tables** — you do not need to verify them line by line; the build already did, and would have blocked the PR. Do glance for *extraction* errors: a row or column the PDF reader misread. A machine cannot catch that.
- [ ] **The one-line summaries in `_manifest.yaml`.** Do they say what the section is about, well enough that someone searching would find it?
- [ ] Wrong? Edit the `.md` or `.yaml` directly in the web UI, or comment and ask the author.

Merging this pull request is the expert sign-off. It still does not publish anything.

---

9. Publish

```
kb publish            # picks the right mode
kb publish --pr       # explicit: open or update a pull request on the hub
```

```
kb publish --direct   # explicit: commit straight to the hub
```

Publishing mirrors your whole store (all four layers) into `federation/<repo-id>/` on the hub and rebuilds the hub's aggregate index.

Mode	When you want it
`--pr`	Normal. Needs a git remote and a `gh` that can open a PR on that host.
`--direct`	A purely local hub — a directory on disk with no remote. Refused on a governed hub that has a remote.
Neither flag	Direct only for a hub with no remote; otherwise a PR, or a clear refusal.

Only artefacts are published. Documents, indexes, manifests and assets go; configuration files and dotfiles stay behind and are named in a warning. That is deliberate: your `hub:` setting may contain a token, and mirroring it would publish a credential.

> If `kb doctor` on the hub names files inside an entry that a publish would never write, an older tool version put them there. Publish again to strip them, and **rotate anything secret they contained** — removing a file does not remove it from Git history.

9.1 Publishing from CI

The scaffolded `kb-publish.yml` runs `kb ci-publish` from your own GitHub Actions job. It authenticates to the hub's intake service with GitHub Actions OIDC, so there is no long-lived secret to store or rotate. `kb publish` creates and pushes the tag that triggers it. `dev` and `ba` repos publish by the same route, from their own `kb-code.yml` and `kb-publish.yml`.

9.2 When a publish is refused

Message	What to do
`gh` cannot open a pull request	Install and authenticate `gh` for that host (`GH_HOST=<host>` for Enterprise) until `gh repo view` works inside the hub clone
direct push refused on a governed hub	Use `--pr`, or `kb ci-publish` from CI. The refusal is the governance working.
`federation cycle detected`	A hub chain would loop content back to itself. Check `hub:` on each hub in the chain.
authentication error on every hub command	Check `git --version` — below 2.31 with a credentialed hub URL, nothing will work
intake: the repo "is not registered"	The repo is missing from `federation/registry.yaml` — see §9.3

9.3 Registering publishers on the hub (hub operator)

The intake accepts a publish only from a repository listed in the hub's `federation/registry.yaml`, and that entry — never the payload — decides which `repo_id` it may write. Add every publisher by a normal PR:

```
repos:
 acme/safety-docs: {repo_id: safety, kind: kb}            # a child: Document menu
 acme/payments: {repo_id: payments, kind: dev}            # a dev repo: -code and -svc, Code menu
 acme/requirements:                                       # a ba repo: -plan, Ticket menu
   repo_id: acme-requirements
   kind: ba
```

- `kind` is `kb`, `ba` or `dev` and decides the hub menu (Document, Ticket, Code). The registry wins: a publish whose repo kind maps to something else is refused (the intake answers `403` and names both kinds). An entry without `kind` still works, and the repo's own kind is recorded. On an existing hub, `kb init --kind hub` fills the missing kinds and asks first; it never commits. Upgrade the hub and every local or direct publisher before you commit `kind:` — an older strata-kb rejects the registry. Intake-route publishers are unaffected.
- The `repo_id` must equal `repo_id:` in that repo's `.kb/config.yaml`. It also names the published documents: a `ba` repo with `repo_id: acme-requirements` publishes `acme-requirements-plan`.
- The registry key is the repo's `owner/repo`, mapped to its `repo_id`. Leave `workflow:` unset for tag-triggered publishing (every `ba` publish, and `child` repos using `kb publish`): the intake compares it to the full `job_workflow_ref`, which ends in `@refs/tags/kb-publish/<timestamp>`, so a pin never matches and every publish would be refused. The tag push `kb-publish/*` starts `.github/workflows/kb-publish.yml`.
- Intake accepts only `refs/tags/kb-publish/*`. A `ba` repo therefore publishes when the BA runs `kb publish`, never automatically on merge.
- The BA also sets `intake: <url>` in the repo's `.kb/config.yaml` (an existing `ba` repo adds it by hand and re-runs `kb init --kind ba`).

A published `<repo>-plan` document appears in the federation like `-code`: in the aggregate index, to `kb query` and to MCP (tagged `plan, generated`), and under the **Ticket** menu of the web UI (§10.2). The hub PR it opens is reviewed and merged like any other. If a PR is closed or merged and its `publish/<repo-id>` branch deleted, the next publish opens a fresh PR; this fix ships with 1.17.0, so **redeploy the intake service** after upgrading.

---

10. Running the hub service

One process serves agents, programs and people.

```
STRATA_KB_HTTP_TOKEN=<secret> \
 python -m strata_kb.mcp --hub . --transport http
```

Surface	For	Auth
`http://host:8321/mcp`	AI agents	Bearer token
`http://host:8321/api/…`	scripts and integrations	Bearer token or cookie
`http://host:8321/ui`	people	sign in with a hub account (§10.3); the token opens the UI read-only

A hub is mandatory; the server refuses to start without one and searches only `federation/` — never its own working `.kb/`.

Auto-pull. The HTTP server runs `git pull --ff-only` on the hub checkout every 5 minutes, so a merged hub PR becomes searchable on the next pull without a restart. Change the period with `STRATA_KB_HUB_PULL_INTERVAL` (seconds; `0` turns it off). A hand-edited or diverged checkout is left alone: the pull fails, a warning is logged, and the server keeps serving what it has.

10.1 Docker

```
kb docker-setup       # hub: writes .env, generates a token, starts the service
docker compose up -d
docker compose run --rm hub kb ingest source/handbook.pdf --id hr-handbook
```

On a child, `kb docker-setup` pulls the ingest image so ingestion needs no local Python at all.

10.2 The Ticket menu

The web UI has a **Ticket** menu beside Document. It shows what `ba` repos publish as `<repo>-plan` documents, with a status per story derived from the hub alone: **done** (the id is in a `hist.*` row of the Dev repo's `-svc`), **ready** (its ticket is published), **not ready** (no published ticket yet) or **blocked** (with reasons). Nothing new is stored and there is no write path.

Route	Shows
`/ui/tickets`	Every BA repo and its missions, with status counts. `?repo=<id>` for one repo
`/ui/tickets/{mission}?repo=<id>`	The mission summary and backlog table
`/ui/tickets/{mission}/{story}?repo=<id>`	The ticket. A ticket with no mission is `/ui/tickets/tickets/{story}?repo=<id>`

The repo is a query parameter because nested federation ids contain `/`. Without `?repo=` the one repo that holds the id is used; if several do, the page is a 404 naming them. `*-plan` documents appear under Ticket, not Document, and the overview counts per kind; a search hit inside one links to its Ticket page. The CLI twin is `kb mission next --from-hub [--repo <ba-repo>]`.

10.3 Hub accounts, the Registry page and auto-merge

People sign in to `/ui` with a username and password. Accounts live in `<hub>/.kb-auth/` (`users.db`, `session.key`). Never commit that directory: a new hub's `.gitignore` lists it and the running hub also git-excludes it. Deleting `session.key` logs everyone out.

- **Create the first admin** by opening the hub in a browser: while no account exists it shows `/ui/setup`, and you enter the one-time setup code the hub prints to its log (`docker compose logs hub`) plus a username and password. That account becomes the `admin`; the code is kept in memory only and changes on every restart. Alternatively run `docker compose exec hub kb user add <name> --role admin`, or set `STRATA_KB_BOOTSTRAP_ADMIN` and `STRATA_KB_BOOTSTRAP_PASSWORD_HASH='<hash>'` (the output of `kb user hash`) in `.env`. Keep the single quotes: Docker Compose expands `$` in `.env`, and the hash contains several. The pair applies only while no account exists; a plaintext password is never accepted.

- **Other commands:** `kb user passwd|role|remove|list|hash`. Roles are `admin`, `ba`, `dev`; only `admin` sees and uses the Registry page.
- **The bearer token** still serves `/api`, `/mcp` and CI. In the UI it is read-only and never opens the Registry page.
- **Registry page** (`/ui/registry`). An admin adds or removes a child repo and switches its **auto-merge** on or off. Each edit is a commit on the hub's default branch, pushed with the intake GitHub App and never force-pushed. A protected default branch must allow the App to push. The page is read-only when intake is not configured or the hub has no remote.
- **Auto-merge** merges a repo's whole publish PR right after intake opens it — including any `-svc` content riding along. The App needs `contents: write` and `pull_requests: write`. If the merge fails the PR stays open and the publish is still `done` (`merged: false`); `kb publish` and `kb ci-publish` print "merged" when it merged (the CI step summary still shows `hub PR: <url>`).

10.4 Before you expose it

- **Replace the generated token.** `kb docker-setup` generates one for convenience. Treat it as a placeholder.
- **Set the trusted-proxy count if you are behind a reverse proxy.** The default ignores forwarded headers. Left unset behind a proxy, every caller looks like the proxy, and one caller can exhaust the rate limiters for everyone.
- **Know the limit.** Authentication is a bearer token for `/api`, `/mcp` and CI plus local hub accounts for the UI — no OAuth, no SSO. That is fine on an internal network and not ready for the public internet.

Full deployment guide: `docs/deploy-remote-mcp.md`.

10.5 Ticket Q&A

A Dev who meets a business question about a published ticket asks the BA; the BA answers; the hub keeps the thread. Nothing to configure: the questions live in SQLite at `<hub root>/.kb-qa/qa.db` (in Docker `/data/.kb-qa/qa.db`), created, and git-excluded, on the first Q&A call. Never commit `.kb-qa/`: a new hub's `.gitignore` lists it, and the hub also git-excludes it when the web UI starts. Back it up with the rest of the hub volume.

- **MCP tools.** `kb_question_ask`, `kb_question_list` and `kb_question_answer`. All three, the list included, work only on the hub itself: this hub's HTTP server, or the stdio server run inside the hub's own checkout (`--kb <hub>/.kb`). Any other target is refused: a dev repo's stdio server, or a hub URL read through a local cache clone, answers `Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`. BA and Dev repos point MCP at the HTTP server with `kb mcp-setup`.
- **Who may write.** Any caller the hub authenticates: the bearer token, or a session of any role. `asked_by` and `answered_by` are names the caller types, so they track who said what and authenticate nobody. At most 50 questions per ticket can be open.
- **The Q&A menu.** The left rail's **Q&A** (`/ui/questions`) is read-only. It lists the tickets that have questions, grouped by BA repo, with open, waiting and answered counts (`?all=1` includes fully answered tickets); `/ui/questions/ticket?repo=<repo-id>&ticket=<id>` shows one ticket's questions. The ticket page of the Ticket menu gains an "N unresolved questions" link while any are open or waiting.
- **States.** *Open* until answered. An answer that changes the ticket stays *waiting for publish* until the BA's new ticket revision (its own `> Revision: N`) is on the hub, then it is *answered*.

---

11. Searching

```
kb query "parental leave eligibility"
kb query "parental leave" --tags hr --budget 600
kb get hr-handbook 4.12 --level l2
kb get hr-handbook 4.12 --level l3
kb tags
```

Flag	Effect
`--tags`	Restrict to documents carrying those tags. A document id works as a tag too.
`--budget`	Token ceiling. Advisory — the first result always comes back whole, and sections are never cut mid-way.
`--semantic`	Force the semantic leg; warns if embeddings are not installed

Every result cites `<repo-id>:<doc-id> §<section> (<revision>)`, so you always know which repository, document and edition answered.

Query in the document's own language. The store never translates, so an English query against a Vietnamese document will match little.

---

12. Keeping the hub healthy

```
kb doctor      # structural health; exits non-zero on a real problem
kb reindex     # rebuild federation/index.yaml when it has drifted
kb stats       # token size per layer, per document
```

`kb doctor` reports broken contents, manifest and file disagreements, content modified directly on the hub rather than through a publish, files a publish would never have written, a hub clone storing a credential in its own Git config, and a Git older than the 2.31 floor.

Run it after an upgrade and before a first publish following one. A store that was green can go red on upgrade with nothing having changed on disk — the checks got stricter, and the message tells you which command fixes it.

---

13. Troubleshooting

Symptom	Cause	Fix
`kb build`: table mismatch	A table was edited, added, duplicated or reordered in the summary	Copy the table verbatim from `.raw.md` over the one in `.md`
`kb build`: sections still pending	Summarization did not finish or was skipped	`kb status`, then `kb summarize`, or `/kb-summarize` with no LLM CLI
`kb build`: "L3 changed after summarization"	The document was re-ingested under an existing summary	Re-run `kb summarize` for the affected sections
First `kb ingest` takes 10–30 minutes	The layout model is downloading	Wait once; later runs use the cache
`kb: command not found`	Virtualenv not active	`source .venv/bin/activate`
`kb query` returns nothing	No tag or content match, or the budget is too small	Drop `--tags`, raise `--budget`, and query in the document's language
Query returns nothing after a merge	Merged on your repo, not yet on the hub	`kb publish`, then merge the hub PR
Every hub command fails on auth	Git older than 2.31 with a credentialed hub URL	Upgrade Git, or use an ssh hub URL
MCP server will not start: `python` not found	The client config calls `python`, your system has only `python3`	Point the command at an absolute interpreter path

14. Command summary

Command	Purpose
`kb init [--kind hub|child]`	Create or refresh the repository
`kb docker-setup`	Prepare Docker: hub starts the service, child pulls the image
`kb ingest <pdf> --id <id>`	Split a document, write L3, scaffold L1/L2
`kb l3 report|confirm <doc>`	Review and confirm flagged L3 sections and AI fixes
`kb l3 lint <doc> --pdf <pdf>`	Check the L3 layout rules R1 to R8
`kb l3 fix on|off|pages|apply`	The L3 AI fix: switch, page images, check and write drafts
`kb status`	What is still pending
`kb summarize <doc>`	Fill or re-fill summaries
`kb build [--strict] [--allow-pending]`	Validate the store
`kb approve <doc>`	Record expert sign-off per section
`kb publish [--pr|--direct]`	Mirror to the hub
`kb query <text>`	Search the hub
`kb get <doc> <section>`	Fetch one section
`kb tags`	List published tags
`kb stats`	Token size per layer
`kb doctor`	Health check
`kb reindex`	Rebuild the aggregate index

Exit codes: `0` success, `1` error or misconfiguration, `2` a citation (or the hub cache) is stale.